Fourier Series and CFT Exam Preparation

Classical 25–30 Minute Problems with Solutions

Contents

1 Introduction **3**

2 Harmonic Band-Pass Filtering **4**
2.1 Problem . . . 4
2.2 Solution . . . 4
2.3 What This Tests . . . 4

3 Keep the K Most Energetic Harmonics **5**
3.1 Problem . . . 5
3.2 Solution . . . 5
3.3 Exam Variations . . . 5

4 Fourier Reconstruction at Specific Times **6**
4.1 Problem . . . 6
4.2 Solution . . . 6
4.3 Important Note . . . 6

5 Calculate Fourier Spectrum **7**
5.1 Problem . . . 7
5.2 Solution . . . 7
5.3 Possible Exam Twist . . . 7

6 Low-Pass Fourier Reconstruction **8**
6.1 Problem . . . 8
6.2 Solution . . . 8
6.3 Why This Is Highly Important . . . 8

7 CFT Magnitude Thresholding **9**
7.1 Problem . . . 9
7.2 Solution . . . 9
7.3 Very Important Pattern . . . 9

8 CFT Low-Pass Filter **10**
8.1 Problem . . . 10
8.2 Solution . . . 10
8.3 Key Insight . . . 10

9 CFT Band-Pass Filter **11**
9.1 Problem . . . 11
9.2 Solution . . . 11
9.3 Visual Summary . . . 11

10 CFT Spectrum Energy **12**
10.1 Problem . . . 12
10.2 Solution . . . 12
10.3 Conceptual Connection . . . 12

11 Reconstruction Error After CFT Filtering **13**
11.1 Problem . . . 13
11.2 Solution . . . 13
11.3 Note . . . 13

12 CFT Reconstruction With Only Strong Frequencies **14**
12.1 Problem . . . 14
12.2 Solution . . . 14
12.3 What This Combines . . . 14

13 Critical Patterns to Memorize **15**
13.1 Fourier Coefficient . . . 15
13.2 Fourier Reconstruction . . . 15
13.3 Harmonic Energy . . . 15
13.4 Fourier Pruning . . . 15
13.5 MSE . . . 15
13.6 CFT Magnitude . . . 15
13.7 Frequency-Domain Masking . . . 16
13.8 Circular Frequency Distance . . . 16

14 Priority Practice Order **17**
14.1 Final Key Observation . . . 17
14.2 Core Methods to Make Automatic . . . 17

Problem 1 Introduction

This document contains **11 classical exam problems** covering both Fourier Series (Task 1/Section A) and Continuous Fourier Transform (Task 2) topics. These problems are designed to be realistic 25–30 minute exam questions that test your ability to:

- Reuse existing Fourier coefficient and reconstruction code
- Add small variations involving filtering, energy pruning, and error calculation
- Apply similar concepts across both 1D (Fourier Series) and 2D (CFT) domains

Key Pattern: Most exam questions follow the philosophy of taking your existing implementation and adding a small operation around it.

Problem 2 Harmonic Band-Pass Filtering

Problem

Modify the `FourierEpicycles` class by implementing:

```
def filter_harmonics(self, min_harmonic, max_harmonic):
```

Keep only harmonics satisfying:

$$\text{min_harmonic} \leq |n| \leq \text{max_harmonic}$$

Set all other coefficients to zero.
Then implement:

```
def calculate_reconstruction_error(self):
```

which returns the MSE between the original signal and the filtered reconstruction.
Example:

```
filter_harmonics(5, 20)
```

should keep:

$$n = -20, \ldots, -5, 5, \ldots, 20$$

but remove:

$$n = -4, \ldots, 4$$

and all harmonics with $|n| > 20$.

Solution

```
def filter_harmonics(self, min_harmonic, max_harmonic):
    if not self.coeffs:
        self.calculate_all_coefficients()

    for n in self.coeffs:
        if abs(n) < min_harmonic or abs(n) > max_harmonic:
            self.coeffs[n] = 0.0
```

```
def calculate_reconstruction_error(self):
    reconstructed = self.approximate(self.t)
    error = np.abs(self.signal - reconstructed) ** 2
    return np.mean(error)
```

What This Tests

- Dictionary traversal
- Harmonic index handling
- Setting coefficients to zero
- Reconstruction using existing method
- MSE calculation for complex signals

Problem 3 Keep the K Most Energetic Harmonics

This is an even more direct variation of Section A's question.

Problem

Implement:

```
def retain_top_k(self, k):
```

Keep only the k harmonics having the largest energy:

$$E_n = |c_n|^2$$

Set all other coefficients to zero.
Return:

- number of retained harmonics
- total retained energy

Solution

```
def retain_top_k(self, k):
    if not self.coeffs:
        self.calculate_all_coefficients()

    energies = {
        n: np.abs(c) ** 2
        for n, c in self.coeffs.items()
    }

    sorted_harmonics = sorted(
        energies.items(),
        key=lambda item: item[1],
        reverse=True
    )

    retained = sorted_harmonics[:k]
    retained_energy = 0.0

    for i, (n, energy) in enumerate(sorted_harmonics):
        if i < k:
            retained_energy += energy
        else:
            self.coeffs[n] = 0.0

    return k, retained_energy
```

Exam Variations

- **Keep the smallest number of harmonics whose total energy is at least r of the total.**
- **Keep exactly k dominant harmonics.**

Both patterns are important to know.

Problem 4 Fourier Reconstruction at Specific Times

Problem

Implement:

```
def reconstruction_values(self, times):
```

which receives a NumPy array of arbitrary time points and returns the reconstructed complex signal at those points.

Also calculate the maximum reconstruction error:

```
def max_error(self, times):
```

Solution

```
def reconstruction_values(self, times):
    result = np.zeros_like(times, dtype=complex)

    for n in self.coeffs:
        result += (
            self.coeffs[n]
            * np.exp(1j * n * self.omega * times)
        )

    return result
```

```
def max_error(self, times):
    reconstructed = self.reconstruction_values(times)
    original = self.signal
    return np.max(np.abs(original - reconstructed))
```

Important Note

Your existing `approximate()` method already does almost exactly this:

```
f_hat = complex(0, 0)
for n in range(-self.N, self.N + 1):
    f_hat += self.coeffs[n] * np.exp(1j * n * self.omega * t)
```

So for questions like "Calculate reconstruction at $t = [\dots]$", **reuse your `approximate()` logic**.

Problem 5 Calculate Fourier Spectrum

Problem

Implement:

```
def get_magnitude_spectrum(self):
```

which returns two arrays:

- harmonics
- magnitudes

where:

$$m_n = |c_n|$$

Then plot the magnitude spectrum.

Solution

```
def get_magnitude_spectrum(self):
    if not self.coeffs:
        self.calculate_all_coefficients()

    harmonics = np.array(list(self.coeffs.keys()))
    magnitudes = np.array([
        np.abs(c)
        for c in self.coeffs.values()
    ])

    return harmonics, magnitudes
```

Plot:

```
def plot_spectrum(self):
    harmonics, magnitudes = self.get_magnitude_spectrum()
    plt.stem(harmonics, magnitudes)
    plt.xlabel("Harmonic n")
    plt.ylabel("|c_n|")
    plt.title("Fourier Magnitude Spectrum")
    plt.show()
```

Possible Exam Twist

Instead of magnitude, they may ask for **energy spectrum**:

$$\text{energy} = |c|^2$$

Remember:

$$\text{magnitude} = |c| \quad \text{and} \quad \text{energy} = |c|^2$$

Problem 6 Low-Pass Fourier Reconstruction

Problem

Implement:

```
def low_pass(self, K):
```

Keep only harmonics:

$$-K \leq n \leq K$$

Set all other coefficients to zero.
Then calculate the MSE.

Solution

```
def low_pass(self, K):
    if not self.coeffs:
        self.calculate_all_coefficients()

    for n in self.coeffs:
        if abs(n) > K:
            self.coeffs[n] = 0.0
```

MSE:

```
def mse(self):
    reconstructed = self.approximate(self.t)
    return np.mean(np.abs(self.signal - reconstructed) ** 2)
```

Why This Is Highly Important

This is probably one of the **most exam-friendly variations** of your Section A question.
They can easily say:

> "Implement a low-pass Fourier filter that retains only the first K harmonics and evaluate the reconstruction error."

You should immediately think:

```
for n in self.coeffs:
    if abs(n) > K:
        self.coeffs[n] = 0
```

Problem 7 CFT Magnitude Thresholding

Now switch to Task 2 (2D Continuous Fourier Transform).

Problem

After computing the CFT, implement:

```
def threshold(self, real, imag, threshold_value):
```

Set all frequency components whose magnitude is below the threshold to zero:

$$|F(u,v)| < T \quad \text{should become zero}$$

Return the filtered real and imaginary components.

Solution

```
def threshold(self, real, imag, threshold_value):
    real = real.copy()
    imag = imag.copy()

    magnitude = np.sqrt(real**2 + imag**2)
    mask = magnitude < threshold_value

    real[mask] = 0.0
    imag[mask] = 0.0

    return real, imag
```

Very Important Pattern

You should recognize:

```
magnitude = np.sqrt(real**2 + imag**2)
```

or equivalently:

```
magnitude = np.abs(real + 1j * imag)
```

Then:

```
mask = magnitude < threshold
real[mask] = 0
imag[mask] = 0
```

Problem 8 CFT Low-Pass Filter

Problem

Implement:

```
def low_pass(real, imag, cutoff):
```

Keep only frequencies inside a circular region around the center:

$$\sqrt{(i - c_x)^2 + (j - c_y)^2} > \text{cutoff} \quad \text{should become zero}$$

Solution

```
def low_pass(self, real, imag, cutoff):
    rows, cols = real.shape
    cx = rows // 2
    cy = cols // 2

    real = real.copy()
    imag = imag.copy()

    for i in range(rows):
        for j in range(cols):
            distance = np.sqrt((i - cx)**2 + (j - cy)**2)
            if distance > cutoff:
                real[i, j] = 0.0
                imag[i, j] = 0.0

    return real, imag
```

Key Insight

This is basically the **opposite of your given high-pass filter**.

Your existing code:

```
if distance <= cutoff:
    real[i, j] = 0
    imag[i, j] = 0
```

Low-pass:

```
if distance > cutoff:
    real[i, j] = 0
    imag[i, j] = 0
```

That's it.

Problem 9 CFT Band-Pass Filter

Problem

Keep only frequencies satisfying:

$$\text{low_cutoff} \leq \text{distance} \leq \text{high_cutoff}$$

where distance is measured from the center of the spectrum.

Solution

```
def band_pass(self, real, imag, low_cutoff, high_cutoff):
    rows, cols = real.shape
    cx = rows // 2
    cy = cols // 2

    real = real.copy()
    imag = imag.copy()

    for i in range(rows):
        for j in range(cols):
            distance = np.sqrt((i - cx)**2 + (j - cy)**2)

            if distance < low_cutoff or distance > high_cutoff:
                real[i, j] = 0.0
                imag[i, j] = 0.0

    return real, imag
```

Visual Summary

Filter Type	Condition
Low-Pass	$0 \leq \text{distance} \leq \text{high}$
High-Pass	$\text{low} \leq \text{distance} \leq \infty$
Band-Pass	$\text{low} \leq \text{distance} \leq \text{high}$

Problem 10 CFT Spectrum Energy

Problem

After obtaining `real` and `imag`, calculate the total energy of the frequency spectrum:

$$E = \sum_{u,v} |F(u,v)|^2$$

Also return the percentage of energy contained inside a circular cutoff.

Solution

```
def total_energy(real, imag):
    magnitude_squared = real**2 + imag**2
    return np.sum(magnitude_squared)
```

Energy inside cutoff:

```
def energy_inside_cutoff(real, imag, cutoff):
    rows, cols = real.shape
    cx = rows // 2
    cy = cols // 2

    energy = 0.0

    for i in range(rows):
        for j in range(cols):
            distance = np.sqrt((i - cx)**2 + (j - cy)**2)
            if distance <= cutoff:
                energy += real[i, j]**2 + imag[i, j]**2

    return energy
```

Energy ratio:

```
ratio = inside_energy / total_energy
```

Conceptual Connection

This is conceptually **very similar to Section A's energy pruning**, except now the energy belongs to (u, v) frequency components instead of harmonic n.

Problem 11 Reconstruction Error After CFT Filtering

Problem

After reconstructing an image using `InverseCFT2D`, calculate the MSE between the original image and reconstructed image.

Implement:

```
def calculate_mse(original, reconstructed):
```

Solution

For real images:

```
def calculate_mse(original, reconstructed):
    error = (original - reconstructed) ** 2
    return np.mean(error)
```

For complex reconstruction (general case):

```
def calculate_mse(original, reconstructed):
    error = np.abs(original - reconstructed) ** 2
    return np.mean(error)
```

Note

For your Task 2, `reconstruct()` returns the real part, so the first version is sufficient.

Problem 12 CFT Reconstruction With Only Strong Frequencies

Problem

Compute the CFT, remove all frequency components whose magnitude is less than r times the maximum magnitude, reconstruct the image, and calculate MSE.

For example:

$$r = 0.01$$

means:

$$\text{keep frequencies where } |F(u,v)| \geq 0.01 \cdot \max(|F|)$$

Solution

```
real, imag = cft.compute_cft()

magnitude = np.sqrt(real**2 + imag**2)
threshold = 0.01 * np.max(magnitude)

mask = magnitude < threshold

real[mask] = 0.0
imag[mask] = 0.0

inverse = InverseCFT2D(
    real,
    imag,
    cft.u,
    cft.v,
    img.x,
    img.y
)

reconstructed = inverse.reconstruct()

mse = np.mean((img.image - reconstructed) ** 2)
```

What This Combines



This is a **very good practice problem** because it combines all the key concepts.

Problem 13 Critical Patterns to Memorize

Fourier Coefficient

```
integrand = signal * np.exp(-1j * n * omega * t)
cn = (1 / T) * np.trapezoid(integrand, x=t)
```

Fourier Reconstruction

```
result = 0
for n in coeffs:
    result += coeffs[n] * np.exp(1j * n * omega * t)
```

Harmonic Energy

```
energy = np.abs(c) ** 2
```

Fourier Pruning

```
energies = {
    n: np.abs(c)**2
    for n, c in coeffs.items()
}

sorted_energies = sorted(
    energies.items(),
    key=lambda x: x[1],
    reverse=True
)
```

Then either:

- keep top K
- keep until cumulative energy $\geq$ target

MSE

For complex signal:

```
np.mean(np.abs(original - reconstructed) ** 2)
```

For real image:

```
np.mean((original - reconstructed) ** 2)
```

CFT Magnitude

```
magnitude = np.sqrt(real**2 + imag**2)
```

Frequency-Domain Masking

```
mask = magnitude < threshold
real[mask] = 0
imag[mask] = 0
```

Circular Frequency Distance

```
distance = np.sqrt((i - cx)**2 + (j - cy)**2)
```

Then:

- distance $\leq$ cutoff $\rightarrow$ low frequencies
- distance $>$ cutoff $\rightarrow$ high frequencies

Problem 14 Priority Practice Order

If you only have limited time, practice these **in this order**:

Priority	Problem	Difficulty
1	Top-K Fourier harmonic pruning	Easy
2	Energy-ratio Fourier pruning	Easy
3	Fourier low-pass reconstruction	Easy
4	Fourier reconstruction MSE	Very easy
5	CFT magnitude thresholding	Easy
6	CFT low-pass filtering	Easy
7	CFT band-pass filtering	Medium
8	CFT filtered-image MSE	Easy
9	Frequency-domain energy calculation	Medium
10	Strong-frequency CFT reconstruction	Medium

Final Key Observation

The Section A online question didn't introduce a new Fourier concept. It took your existing Fourier-series implementation and added:

- Energy calculation
- Sorting
- Reconstruction error

Your next exam question will very likely follow the same philosophy: **reuse your existing code and add a small operation around them**.

Core Methods to Make Automatic

- `calculate_cn`
- `calculate_all_coefficients`
- `approximate`
- CFT integration
- Inverse-CFT code
- Separable trapezoidal integration

Make these completely automatic before tomorrow.